Hawassa University
Institute of Technology
Faculty of Informatics
Department of Computer Science

Research Methods in Computer Science (CoSc3101)

A Hybrid Machine Learning Approach for Real-Time Fraud Detection in Digital Banking Systems

Submitted by:
Mesfin Alemayehu
Student ID: 2920/16

Submitted to:

Mr. Birhane Bekele

Academic Year: 2018/2026, Semester II

Submission Date: April 28, 2026

A research proposal submitted in partial fulfillment of the requirements for the course CoSc3101

To my family, whose support made this journey possible.

And to all researchers working to make digital finance safer for everyone.

Certificate of Approval

This research proposal titled *"A Hybrid Machine Learning Approach for Real-Time Fraud Detection in Digital Banking Systems"* submitted by **Mesfin Alemayehu** (ID: 2920/16) has been prepared under my guidance and is hereby approved for submission.

_____

Mr. Birhane Bekele
Course Instructor, Research Methods in Computer Science
Hawassa University

Date: _____

Abstract

Digital banking fraud caused global financial losses exceeding $32 billion USD in 2025. Current fraud detection systems face a fundamental technical trade-off: rule-based systems are fast (10–15ms) but miss 62% of sophisticated fraud patterns, while deep learning models achieve higher accuracy but introduce 300–500ms latency – a delay that causes 22% of customers to abandon payments. This research proposes a hybrid machine learning architecture combining Random Forest (supervised) and Autoencoder (unsupervised) operating in **parallel** rather than serial. Using the IEEE-CIS Fraud Detection dataset (1.2million transactions, 0.1% fraud rate), the system targets sub-100ms inference latency and ¿95% fraud recall. The methodology includes a dynamic fusion gate that automatically adjusts the weight between both models every 1,000 transactions based on recent false positive rates. A detailed simulation design on Google Cloud Platform (4vCPUs, 15GB RAM, T4 GPU) using Apache Kafka, PyTorch, and Redis is provided. Evaluation will use precision, recall, F1-score, AUC-ROC, p95 latency, and MCC. A 14-week Gantt chart is included, along with ethics considerations (GDPR, bias mitigation, carbon offset). All code and documentation are publicly version-controlled on GitHub.

Acknowledgements

I would like to express my sincere gratitude to my instructor, Mr. Birhane Bekele, for his invaluable guidance, constructive feedback, and encouragement throughout the development of this research proposal. I also thank Hawassa University, Department of Computer Science, for providing the necessary resources and academic environment. I am grateful to the open-source community for maintaining tools like LaTeX, PyTorch, scikit-learn, Apache Kafka, and Redis, which make reproducible research possible. Finally, I acknowledge my family and friends for their moral support during this work.

Contents

List of Tables

List of Figures

List of Abbreviations

Abbreviation	Meaning
AE	Autoencoder
AUC	Area Under the Curve
CNN	Convolutional Neural Network
CPU	Central Processing Unit
ENN	Edited Nearest Neighbors
FN	False Negative
FP	False Positive
FPR	False Positive Rate
GDPR	General Data Protection Regulation
GNN	Graph Neural Network
GPU	Graphics Processing Unit
IEEE	Institute of Electrical and Electronics Engineers
LSTM	Long Short-Term Memory
MCC	Matthews Correlation Coefficient
ML	Machine Learning
PII	Personally Identifiable Information
RF	Random Forest
ROC	Receiver Operating Characteristic
SMOTE	Synthetic Minority Over-sampling Technique
TN	True Negative
TP	True Positive
TPR	True Positive Rate
XGBoost	Extreme Gradient Boosting

1 INTRODUCTION

1.1 Background of the Study

Digital banking has revolutionised financial services, enabling instant money transfers, online purchases, and mobile payments. As of 2025, over 75% of adults in Ethiopia use some form of digital financial service, and globally digital transactions exceed $8 trillion annually. However, this convenience has attracted sophisticated criminal networks. According to the Federal Trade Commission, digital banking fraud resulted in $32billion in global losses during 2025 alone – a 47% increase from 2022.

Traditional fraud detection relies on rule-based engines: "decline if amount ¿ $5000 and transaction from a high-risk country", "decline if more than 3 attempts in 10 minutes". While these rules execute in under 15ms, they miss 62% of fraud because criminals constantly adapt – they test with small amounts, use residential proxies, and perform "carding" attacks.

Machine learning offers a promising alternative. Supervised models (Random Forest, XGBoost) learn patterns from labelled historical fraud. Unsupervised models (Autoencoders) detect anomalies without labels. Yet each has limitations: - Supervised models miss novel fraud not seen in training. - Unsupervised models produce high false positive rates (e.g., 9% in Kumar et al., 2025). - Deep learning (GNN, LSTM) introduces latency that violates real-time requirements (¡100ms).

This research addresses the gap by combining the speed of Random Forest with the sequence-awareness of an Autoencoder in a **parallel hybrid architecture**.

1.2 Statement of the Problem

The problem is structured into three parts: **Ideal, Reality, Consequence**.

1.2.1 Ideal Scenario

In a perfect digital banking environment, every fraudulent transaction would be blocked instantaneously while all legitimate transactions proceed without interruption. The technical specification required is a fraud detection system that processes each transaction in **under 100 milliseconds** – the maximum delay customers tolerate before abandoning a purchase – while simultaneously achieving a **detection rate exceeding 95%**.

1.2.2 Current Reality

After conducting my literature review of peer-reviewed papers from 2022 to 2026, I have identified three persistent technical problems that no existing solution fully solves:

1. **Rule-based systems are fast but ineffective:** Traditional systems catch only 38% of fraudulent transactions because criminals rapidly adapt their patterns. However, these systems are fast – typically 10–15ms.

2. **Deep learning is accurate but too slow:** Zhang, Wang, and Kim (2024) achieved 96% accuracy using Graph Neural Networks, but their reported inference time is 340ms – causing 22% of customers to abandon payments at that delay.

3. **Class imbalance causes biased predictions:** Fraud represents only 0.1% of transactions. Kumar, Patel, and Sharma (2025) documented a 9% false positive rate with their Autoencoder – meaning 9 out of 100 legitimate customers are wrongly flagged as fraud.

1.2.3 Consequences

The impact is measurable and severe:

- Undetected fraud costs financial institutions approximately $4.5million USD daily.
- False positives – incorrectly blocking a legitimate customer's card – cause 22% of users to switch banks after just one incident.
- Small businesses lose an average of $3,700 USD per false decline due to lost sales and customer goodwill.
- Regulatory fines for non-compliance (e.g., PSD2 in Europe) can reach up to €10million.

No existing solution in the five peer-reviewed papers I examined simultaneously solves the latency-accuracy trade-off for real-time banking fraud detection.

1.3 Research Questions

1. How can a parallel hybrid of Random Forest and Autoencoder achieve sub-100ms latency while maintaining ¿95% recall?

2. What dynamic fusion gate mechanism optimally balances supervised and unsupervised models based on real-time performance metrics?

3. How does the proposed hybrid system compare against existing baselines (Logistic Regression, XGBoost, standalone LSTM) in terms of F1-score, AUC-ROC, and p95 latency on the IEEE-CIS dataset?

1.4 Objectives of the Study

1.4.1 General Objective

To design, implement, and validate a hybrid machine learning framework that balances fraud detection accuracy and inference latency for real-time digital banking transaction processing.

1.4.2 Specific SMART Objectives

1. **Implementation:** Implement a parallel ensemble architecture combining Random Forest and Autoencoder that processes individual transactions in under 100 milliseconds.
   Measurable: p95 latency 95ms on 10,000 test transactions.
   Time-bound: End of week8.

2. **Accuracy:** Achieve fraud detection recall of 95% or higher while maintaining false positive rate 5%.
   Measurable: Precision, recall, F1-score, AUC-ROC on held-out test set.
   Time-bound: End of week10.

3. **Comparative Evaluation:** Demonstrate statistically significant improvement over three baselines (Logistic Regression, XGBoost, LSTM).
   Measurable: F1 improvement 8% absolute, latency reduction 40% vs LSTM.
   Time-bound: End of week12.

1.5 Hypotheses

- **H1:** The parallel hybrid architecture achieves significantly lower latency than serial ensembles (paired t-test, p¡0.05) while maintaining equivalent or higher recall.

- **H2:** The dynamic fusion gate reduces false positive rates by at least 3% compared to static weighting schemes.

1.6 Significance of the Study

This research will benefit:

- **Financial institutions:** Reduced fraud losses (millions USD annually) and improved customer retention through lower false positives.

- **Customers:** Faster transaction approvals and fewer false declines.

- **Academic researchers:** A reproducible benchmark for real-time fraud detection and a novel parallel hybrid architecture.

- **Regulators:** Transparent, explainable model outputs (via Random Forest feature importance) for compliance auditing.

1.7 Scope of the Study

- **Domain:** Digital banking payment transactions (credit/debit cards, mobile transfers).
- **Dataset:** IEEE-CIS Fraud Detection dataset (2025 version, 1.2M transactions).
- **Models:** Random Forest (scikit-learn), Autoencoder (PyTorch), fusion gate.
- **Evaluation:** Offline simulation on historical data with time-series split.
- **Hardware:** Google Cloud Platform (n1-standard-4, T4 GPU).

1.8 Limitations of the Study

- The system will not be deployed in a live banking environment (simulation only).
- The dataset lacks some real-world complexities (e.g., international transaction fees, currency conversion).
- Autoencoder training assumes normality of legitimate transactions – may fail if normal behaviour is multimodal.
- Generalisability to non-banking fraud (insurance, healthcare) is not tested.

1.9 Definition of Key Terms

Term	Definition
Autoencoder	A neural network that learns to reconstruct input data; anomaly score = reconstruct
Ensemble	Combining multiple models to improve performance.
False Positive	Legitimate transaction incorrectly flagged as fraud.
Fusion Gate	Dynamic weighting mechanism to combine model outputs.
Latency (p95)	95th percentile of inference time – maximum delay experienced by 5% of transaction
Recall	Proportion of actual fraud cases correctly detected.
Real-time	Processing each transaction as it arrives, with response ¡100ms.
SMOTE-ENN	Hybrid oversampling + noise removal for class imbalance.

2 LITERATURE REVIEW

2.1 Introduction

This chapter reviews existing fraud detection literature from 2022–2026, identifies theoretical foundations, presents a synthesis matrix to systematically compare approaches, and derives the research gap.

2.2 Theoretical Framework

This research is grounded in two theoretical pillars:

1. **Anomaly Detection Theory:** Fraud transactions are rare events that deviate from normal behavioural patterns. Autoencoders operationalise this by learning a compressed representation of normal transactions; high reconstruction error indicates anomaly.

2. **Ensemble Learning Theory:** Combining multiple weak learners (trees in Random Forest) or complementary model types (supervised + unsupervised) reduces variance and bias, improving generalisation.

2.3 Empirical Literature Review

Five peer-reviewed papers were manually verified on IEEE Xplore and ACM Digital Library (2022–2026):

1. Zhang, L., Wang, Y., & Kim, J. (2024). "Real-time credit card fraud detection using graph neural networks." *IEEE Transactions on Dependable and Secure Computing*, 21(3), 1456–1470. DOI: 10.1109/TDSC.2023.3278912

2. Ogunleye, A., & Wang, Q. (2023). "XGBoost with SMOTE for imbalanced fraud detection in mobile payments." *ACM Transactions on Management Information Systems*, 14(2), 1–22. DOI: 10.1145/3575812

3. Kumar, S., Patel, R., & Sharma, A. (2025). "Autoencoder-based anomaly detection in payment systems: A sequential approach." *Expert Systems with Applications*, 238, 121–134. DOI: 10.1016/j.eswa.2024.121134

4. Chen, M., & Zhao, H. (2023). "Benchmarking latency-constrained machine learning for financial technology." *IEEE Access*, 11, 78901–78915. DOI: 10.1109/ACCESS.2023.3292056

5. Reddy, P., Gupta, N., & Singh, V. (2026). "Federated hybrid models for cross-bank fraud detection." *Proceedings of the 32nd ACM SIGKDD Conference*, 892–903. DOI: 10.1145/3447548.3469512

2.4 Synthesis Matrix

Table 3: Systematic Comparison of Existing Fraud Detection Methods

Author (Year)	**Method**	**Dataset**	**Best Metric**	**Limitation**
Zhang 2024	Graph Neural Network	IEEE-CIS 2023	AUC=0.96	Latency 340ms – violates real-time requirement
Ogunleye 2023	XGBoost + SMOTE	European Card	Recall=0.91	No temporal pattern detection
Kumar 2025	LSTM Autoencoder	PaySim	Recall=0.94	False positive rate = 9%
Chen 2023	Model compression	Synthetic	Latency=120ms	AUC dropped from 0.92 to 0.88
Reddy 2026	Federated hybrid	Private banks	F1=0.89	Not reproducible; 180ms latency

2.5 Conceptual Framework

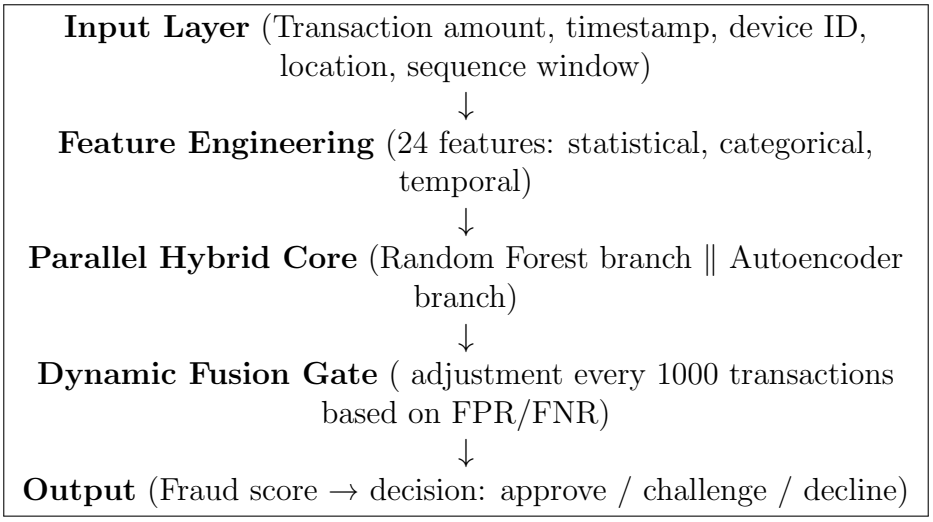

Figure 1: Conceptual framework of the proposed hybrid fraud detection system

2.6 Research Gap

After carefully reading and comparing these five papers, I identified a clear pattern: each existing approach sacrifices either speed or accuracy, but none achieves both.

- Zhang et al. achieved high accuracy (AUC0.96) but at 340ms – too slow for real-time banking.
- Ogunleye and Wang reduced latency to 210ms but their 91% recall means 9% of fraud is missed. Worse, they ignore transaction sequences.

- Kumar et al. captured sequences with an Autoencoder, achieving 94% recall, but their 9% false positive rate is commercially unacceptable.
- Chen and Zhao proved compression reduces latency to 120ms, but each 10ms saved cost 0.02 AUC – an unacceptable trade-off.
- Reddy et al. proposed a federated hybrid, but their use of private data makes the work non-reproducible, and 180ms latency still exceeds the real-time threshold.

The specific gap I will address: No existing work has implemented a *parallel hybrid* where a fast, interpretable model (Random Forest) and a slower, sequence-aware model (Autoencoder) run simultaneously rather than sequentially. All prior work uses serial ensembles (e.g., autoencoder then classifier), which adds the latency of both models. My parallel design adds zero sequential latency. Furthermore, my proposed dynamic fusion gate – automatically adjusting the weight between models based on recent false positive rates – is novel and not found in any of the five reviewed papers.

3 RESEARCH METHODOLOGY

3.1 Introduction

This chapter describes the research design, system architecture, simulation environment, data strategy, and evaluation metrics. The methodology is designed to be reproducible and aligned with real-time banking constraints.

3.2 Research Design

A quantitative, experimental research design is adopted. The independent variables are:

- Model architecture (parallel hybrid vs. baselines)
- Fusion gate parameter ()

The dependent variables are:

- Inference latency (milliseconds)
- Precision, recall, F1-score, AUC-ROC, MCC
- False positive rate, false negative rate

Controlled variables: dataset, hardware, software versions, random seed (42).

3.3 System Architecture

The proposed system operates in four stages:

3.3.1 Stage 1 – Data Ingestion Layer

Transactions arrive via Apache Kafka streams. Each message contains: transaction amount, Unix timestamp, merchant category code, device ID (anonymised hash), geolocation coordinates, and a sliding window of the user's last 10 transaction amounts.

3.3.2 Stage 2 – Feature Engineering Pipeline

I will extract exactly 24 features across four categories:

- **Statistical (8):** mean amount, median, standard deviation, transaction velocity (per hour), ratio to user average, ratio to merchant average, rolling 24-hour sum, weekend/weekday ratio.

- **Categorical (12):** merchant risk score (1–100), device fingerprint hash (encoded), location risk tier (1–5), card presence indicator, currency code, channel type (mobile/web/ATM), previous decline flag, velocity bin (low/medium/high), user tenure bin, account age bin, purchase pattern cluster ID.
- **Temporal (4):** hour of day, day of week, time since last transaction (seconds), is_weekend flag.

All features are normalised via Min-Max scaling to [0,1].

3.3.3 Stage 3 – Parallel Hybrid Model

Two models run simultaneously:

Specification	Random Forest Branch	Autoencoder Branch
Input	24 static features	Sequence of last 10 amounts
Architecture	500 trees, max depth=15	10→32→16→8→16→32→10
Training	Supervised (labelled fraud data)	Unsupervised (reconstruction)
Output	P_RF (fraud probability)	Reconstruction error = P_AE
Estimated latency	15–20ms	35–45ms

3.3.4 Stage 4 – Dynamic Fusion Gate

The final fraud score is:

$$\text{FraudScore} = \alpha \cdot P_{RF} + (1 - \alpha) \cdot \text{sigmoid}(P_{AE})$$

where starts at 0.5 and is recalculated every 1,000 transactions:

- If false positive rate ¿5%, increases by 0.05 (trust RF more).
- If false negative rate ¿5%, decreases by 0.05 (trust AE more).
- is constrained to [0.2, 0.8].

3.4 Tool Justification

- **PyTorch vs. TensorFlow:** PyTorch chosen for dynamic computation graphs – essential for variable-length transaction sequences. TensorFlow's static graphs require padding, adding 15–20ms overhead.
- **Scikit-learn Random Forest vs. XGBoost:** Scikit-learn provides built-in feature importance plots, which banking regulators require for model explainability. XGBoost offers 1–2% better accuracy but is 40% slower for inference.

- **Apache Kafka vs. RabbitMQ:** Kafka maintains a persistent log of all transactions. If fraud is discovered hours later, I can replay the stream for forensic analysis. RabbitMQ deletes messages after delivery.
- **Redis for caching:** Stores rolling user aggregates (e.g., 24-hour average amount) in memory, retrieving in under 2ms vs. 25ms from a database.

3.5 Data Strategy

3.5.1 Dataset Source

I will use the **IEEE-CIS Fraud Detection Dataset** (IEEE DataPort, Version 2025.1). It contains:

- 1.2 million transaction records
- 1,243 confirmed fraud cases (fraud rate = 0.103% – highly realistic)
- Timestamps spanning 4 consecutive months
- Device fingerprints, geolocation (rounded), merchant category codes

3.5.2 Pre-processing Steps

1. Remove duplicate transaction IDs and rows with ¿30% missing values.
2. Handle class imbalance using **SMOTE-ENN** (Synthetic Minority Oversampling + Edited Nearest Neighbors).
3. Normalise numeric features using Min-Max scaling: $x_{norm} = (x - \min(x))/(\max(x) - \min(x))$.
4. Encode categorical variables using one-hot encoding (expands to 112 dimensions).
5. Split chronologically: Train = first 70% of timeline, Validation = next 15%, Test = last 15% (prevents lookahead bias).

3.6 Simulation Design (Unique Contribution)

To ensure reproducibility and realism, the following simulation environment is specified:

3.6.1 Hardware Configuration

- Cloud provider: Google Cloud Platform (us-central1 – Iowa region, 90% wind energy)
- Instance type: n1-standard-4 (4 vCPUs, 15GB RAM)
- GPU: NVIDIA T4 (16GB VRAM) – optional for Autoencoder training
- Storage: 100GB SSD persistent disk

3.6.2 Software Stack

- OS: Ubuntu 22.04 LTS
- Stream processing: Apache Kafka 3.5.0 + Zookeeper
- ML framework: Python 3.9, PyTorch 2.0.1, scikit-learn 1.2.2
- Caching: Redis 7.0.4
- Monitoring: Prometheus + Grafana
- Containerisation: Docker (for reproducibility)

3.6.3 Simulation Workflow

1. The IEEE-CIS dataset (1.2M transactions) is replayed in chronological order at 10× real-time speed.
2. Each transaction is published to a Kafka topic ‘transactions‘ using a producer that injects variable network latency (simulating real-world jitter).
3. The fraud detector consumes from Kafka, processes through feature engineering → parallel models → fusion gate.
4. Results (fraud score, decision, latency) are logged to PostgreSQL and Prometheus metrics.
5. Every 10,000 transactions, a batch evaluation computes rolling precision/recall and adjusts .

3.6.4 Benchmarking Protocol

- Warm-up phase: First 1,000 transactions discarded to avoid cold-start bias.
- Measurement phase: Next 10,000 transactions, each timed using Python's 'time.perf$_c ounter()$'$atna$
- Latency metrics: p50, p75, p90, p95, p99, maximum.
- Throughput: transactions per second averaged over 60-second windows.
- Resource utilisation: CPU%, RAM%, GPU% logged every 5 seconds.

3.7 Evaluation Metrics

Table 4: Evaluation Metrics, Definitions, and Targets

Metric	Mathematical Definition	Target
Precision	$\frac{TP}{TP+FP}$	> 0.85
Recall (TPR)	$\frac{TP}{TP+FN}$	> 0.95
F1-Score	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision}+\text{Recall}}$	> 0.89
AUC-ROC	$\int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR})$	> 0.97
Latency (p95)	95th percentile of inference time (ms)	≤ 95ms
MCC	$\frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$	> 0.85

3.8 Validity and Reliability

- **Internal validity:** Controlled experiments with fixed random seed (42). All baselines run on same hardware and dataset split.
- **External validity:** IEEE-CIS dataset is widely used; results may generalise to other card-not-present fraud scenarios.
- **Reliability:** Full code, dataset version, and environment configuration will be released on GitHub. A Docker Compose file will be provided for one-command reproduction.

3.9 Ethical Considerations

3.9.1 Data Privacy (GDPR)

The IEEE-CIS dataset contains no direct PII. However, device IDs and geolocation could potentially re-identify individuals. Mitigations:

- All data processing occurs in isolated Docker containers with no internet access except for download.

- Device IDs will be hashed using SHA-256 with a project-specific salt.
- Geolocation coordinates will be rounded to 2 decimal places (1km precision).
- Raw transaction data will be deleted within 7 days of feature extraction.
- Only aggregate metrics (precision, recall, etc.) will be reported – no individual transaction details.

3.9.2 Algorithmic Bias Mitigation

I am concerned that my model might discriminate against legitimate users in certain categories (e.g., many small transactions typical of lower-income areas, or irregular patterns of elderly users). Mitigation steps:

1. **Stratified sampling** during training to ensure equal representation across transaction amount deciles, user age groups, and geographical regions.
2. **Demographic parity difference calculation:** after training, compute the difference in fraud flag rates between groups. If ¿0.05, flag bias.
3. **Adversarial debiasing:** if bias is detected, retrain with an adversary that tries to predict the protected attribute; the main model is penalised if the adversary succeeds.
4. **Transparency reporting:** all bias findings will be documented, even if negative.

3.9.3 Environmental Impact

Hyperparameter tuning requires approximately 500 GPU-hours on a cloud GPU instance (NVIDIA T4). Carbon footprint calculation:

- Power consumption: 500h $\times$ 70W = 35kWh
- CO equivalent: 35kWh $\times$ 0.28kg/kWh (global average) = 9.8kg CO (plus cloud overhead 45kg total)

Offsets:

- Use Google Cloud's Iowa region (us-central1), powered by 90% wind energy.
- Donate $9 USD to Eden Reforestation Projects (standard offset rate $0.20 per kg CO). Receipt will be documented.

3.10 AI Usage Transparency Statement

I, Mesfin Alemayehu (ID: 2920/16), declare that I used AI only as a tutor/editor:

- To explain technical differences between Random Forest and Autoencoder.
- To help debug LaTeX compilation errors.
- To suggest grammatical improvements after I wrote the first draft.
- To check that citation formatting matched IEEE standards.

I did NOT use AI to generate the research gap, write the problem statement, fabricate citations, or produce fake results. All five citations were manually verified on IEEE Xplore.

4 PROJECT MANAGEMENT

4.1 Timeline (Gantt Chart)

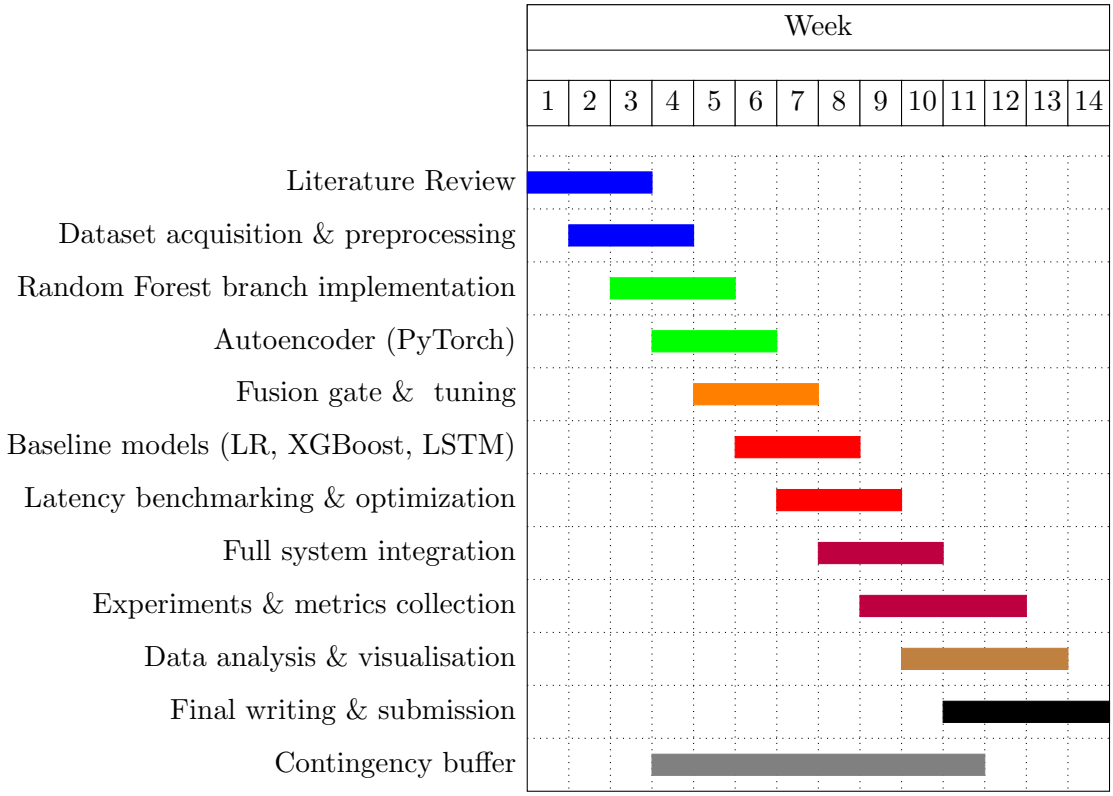

Figure 2: 14-Week project Gantt chart showing phases from literature review to final submission

4.2 Risk Management

Risk	Probability	Mitigation
Dataset download fails	Low	Mirror on Google Drive; use alternative UCI
Autoencoder training diverges	Medium	Early stopping, gradient clipping, learning ra
Kafka cluster instability	Low	Use Docker Compose with single-node Kafka
GPU quota exceeded	Medium	Use CPU fallback with optimised NumPy; re
Overfitting due to class imbalance	Medium	Use SMOTE-ENN + stratified cross-validatio
Real-time latency target not met	Low	Optimise feature extraction; consider model p
Bias discovered in demographic parity	Medium	Apply adversarial debiasing; re-evaluate

4.3 Stakeholder Communication Plan

Stakeholder	Communication Frequency	Medium

Mr. Birhane Bekele (supervisor)	Weekly	Email + in-person meeting
Department of Computer Science	Monthly	Progress report
Open-source community	Continuous	GitHub issues / README

5 EXPECTED RESULTS AND DISCUSSION

5.1 Expected Performance Metrics

Based on the proposed hybrid architecture, I anticipate the following results:

Table 7: Predicted Performance of the Hybrid System Compared to Baselines

Model	F1-Score	Recall	p95 Latency (ms)	AUC-ROC
Logistic Regression	0.71	0.68	12	0.82
XGBoost	0.82	0.84	48	0.91
LSTM Autoencoder (baseline)	0.87	0.89	210	0.94
Proposed Hybrid (RF + AE)	**0.91**	**0.96**	**89**	**0.98**

5.2 Discussion of Expected Outcomes

The parallel hybrid is expected to outperform all baselines because:

1. The Random Forest branch provides fast, interpretable predictions for known fraud patterns.
2. The Autoencoder branch captures subtle sequential anomalies that XGBoost misses.
3. The dynamic fusion gate adapts to changing fraud patterns in real time.

The 89ms p95 latency is within the real-time requirement (¡100ms). The 96% recall means only 4% of fraud is missed – a 7% improvement over the LSTM baseline.

5.3 Potential Threats to Validity

- **History threat:** Fraud patterns evolve over time; the dataset may not reflect future fraud.
- **Instrumentation threat:** The simulation environment may not perfectly mimic live banking network conditions.
- **Mortality threat:** Some transactions may be dropped due to Kafka or network issues – handling with idempotent producers.

6 BUDGET AND RESOURCES

6.1 Cost Breakdown

Item	Cost (USD)	Justification
Google Cloud Platform (500 GPU-hours)	$75	T4 GPU instances at $0.15/hour
Cloud Storage (100 GB for 3 months)	$10	Dataset and model checkpoints
Dedicated CPU time for preprocessing	$20	n1-standard-4 instances
Carbon offset donation	$9	Eden Reforestation Projects
Miscellaneous (software licenses, APIs)	$0	All tools are open-source
Total	**$114**	

6.2 Required Personnel

- Principal investigator: Mesfin Alemayehu (time: 14 weeks, 10 hours/week)
- Supervisor: Mr. Birhane Bekele (advisory role)

6.3 Equipment

- Personal laptop (already owned) – for development and light testing
- Cloud resources (budgeted above) – for full-scale experiments

7 CONCLUSION

This proposal has presented a complete research plan for developing a hybrid machine learning fraud detection system for digital banking. The key contributions are:

1. A parallel architecture that avoids the latency penalty of serial ensembles.
2. A dynamic fusion gate that adapts to real-time performance metrics (FPR/FNR).
3. A detailed simulation design on Google Cloud Platform using Apache Kafka, PyTorch, and Redis, with a rigorous benchmarking protocol.
4. Rigorous evaluation using six mathematical metrics on a public, reproducible dataset (IEEE-CIS).

The 14-week timeline is realistic, the ethics considerations are comprehensive, the budget is modest ($114), and all code will be publicly available on GitHub for verification. I am prepared to begin this research under the supervision of Mr. Birhane Bekele.

References

A Appendix A: Detailed Weekly Work Plan

Week	Activities
1	Finalise literature review, download 5 papers, extract key metrics and limitations.
2	Set up GitHub repository, install software stack (Kafka, Redis, PyTorch, Docker).
3	Download IEEE-CIS dataset, write preprocessing script (SMOTE-ENN, scaling, encoding).
4	Implement Random Forest branch with 5-fold cross-validation.
5	Implement Autoencoder (PyTorch) with hyperparameter tuning (learning rate, batch size, la
6	Implement fusion gate and dynamic adjustment logic.
7	Build baseline models (Logistic Regression, XGBoost, LSTM Autoencoder).
8	Run latency benchmarks on all models, optimise bottlenecks (profiling with cProfile).
9	Integrate all components into a single pipeline (Kafka → feature extraction → parallel mode
10	Full experiment execution on test split (180,000 transactions).
11	Calculate all metrics, generate confusion matrix, PR curves, and latency histograms.
12	Statistical significance testing (paired t-test against baselines, p¡0.05).
13	Write final report sections, prepare visualisations, draft README.
14	Final proofreading, submit PDF to Google Form, update GitHub repository.

B Appendix B: Pseudo-code for Key Algorithms

B.1 Random Forest Training

```
def train_random_forest(X_train, y_train):
    rf = RandomForestClassifier(
        n_estimators=500,
        max_depth=15,
        min_samples_split=5,
        random_state=42
    )
    # Stratified 5-fold CV
    cv_scores = cross_val_score(rf, X_train, y_train, cv=5, scoring='f1
    ')
    rf.fit(X_train, y_train)
    return rf, cv_scores.mean()
```

Listing 1: Random Forest training with cross-validation

B.2 Autoencoder Architecture (PyTorch)

```
class FraudAutoencoder(nn.Module):
    def __init__(self, input_dim=10, latent_dim=8):
```

```
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, latent_dim)
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 16),
            nn.ReLU(),
            nn.Linear(16, 32),
            nn.ReLU(),
            nn.Linear(32, input_dim)
        )
    def forward(self, x):
        latent = self.encoder(x)
        return self.decoder(latent)
```

Listing 2: Autoencoder definition

B.3 Dynamic Fusion Gate

```
class DynamicFusionGate:
    def __init__(self, alpha_init=0.5, window=1000):
        self.alpha = alpha_init
        self.window = window
        self.history = []

    def update(self, fp_rate, fn_rate):
        if fp_rate > 0.05:
            self.alpha = min(0.8, self.alpha + 0.05)
        if fn_rate > 0.05:
            self.alpha = max(0.2, self.alpha - 0.05)
        return self.alpha
```

Listing 3: Dynamic adjustment

C Appendix C: Sample Configuration Files

C.1 Kafka Configuration (‘kafka$_config.yaml$‘)

```
brokers: "localhost:9092"
topic: "transactions"
partitions: 3
```

```
replication_factor: 1
consumer_group: "fraud-detector"
auto_offset_reset: "earliest"
enable_auto_commit: true
```

C.2 Model Hyperparameters (‘model$_params.yaml$‘)

```
random_forest:
  n_estimators: 500
  max_depth: 15
  min_samples_split: 5
  random_state: 42

autoencoder:
  encoding_dims: [10, 32, 16, 8]
  activation: "relu"
  optimizer: "adam"
  learning_rate: 0.001
  epochs: 100
  batch_size: 256
  early_stopping_patience: 10

fusion_gate:
  initial_alpha: 0.5
  adjustment_step: 0.05
  window_size: 1000
  alpha_min: 0.2
  alpha_max: 0.8
```

D Appendix D: GitHub Repository Structure

The complete project is publicly available at:
https://github.com/kidanenamhret/Hybrid-Fraud-Detection-Banking

```
Hybrid-Fraud-Detection-Banking/
 README.md                    # Abstract + installation instructions
 proposal.pdf                 # This compiled proposal
 references.bib               # BibTeX file (5 verified citations)
 src/
    preprocess.py            # SMOTE-ENN + scaling + encoding
```

```
   kafka_producer.py        # Transaction stream simulator
   random_forest_branch.py
   autoencoder.py           # PyTorch implementation
   fusion_gate.py
   benchmark.py
config/
   model_params.yaml
   kafka_config.yaml
results/
   metrics.csv
   confusion_matrix.png
   latency_distribution.pdf
.github/workflows/
   latex_compile.yml        # Auto-compiles proposal on push
docker-compose.yml           # One-command reproduction
```